

> **TECH_MANUAL**

PokeTerminal · Manual Técnico

DOCUMENTACIÓN

01. Inicio

02. Arquitectura

03. Requisitos

04. Instalación

05. Funcionamiento

06. Clases

07. Persistencia

08. Pruebas

09. Errores

10. Mantenimiento

11. UML

12. Versiones

documentation/manual-tecnico.html

Manual Técnico **PokeTerminal**

Documentación técnica de la aplicación de consola PokeTerminal, desarrollada en Java. Describe su organización por paquetes, flujo principal, modelo de clases, persistencia de partidas y mecanismos de interacción.

project: PokeTerminal

language: Java

type: Console

UML: included

Arquitectura del sistema

El proyecto organiza sus responsabilidades en paquetes separados. El punto de entrada crea el controlador principal y desde este se coordinan la interfaz, el mundo, los modelos, las batallas y el almacenamiento.

Entrada / UI

`ui` y `utilidades` manejan presentación, arte ASCII, colores ANSI y lectura de teclado.

Dominio

`modelos` contiene Jugador, Pokémon, Partida, Movimiento, Mochila, Medalla, Gimnasio y entidades relacionadas.

Mundo

`mapas` contiene Mapa, MapaCiudad, MapaAereo y los edificios del mundo.

Lógica

`logica` coordina fórmulas, combates y ejecución de la partida mediante `PoketerminalApp`.

Estructura del proyecto

```
usac.cunoc.ipc1.poketerminal/
├── Main.java
├── logica/
│   ├── Formulas.java
│   ├── SistemaBatalla.java
│   └── PoketerminalApp.java
├── modelos/
│   ├── Pokemon.java · Jugador.java · Partida.java
│   ├── Movimiento.java · Mochila.java · Medalla.java
│   ├── Gimnasio.java · EntrenadorNPC.java
│   ├── Especie.java · Posicion.java · EstadisticasPartida.java
├── mapas/
│   ├── Mapa.java · MapaCiudad.java · MapaAereo.java
```


El material proporcionado no incluye una especificación de versión del JDK ni scripts de compilación. Por eso este manual no inventa una versión mínima oficial.

Instalación y ejecución

1. Ubicar el código fuente respetando la estructura de paquetes.
2. Abrir el proyecto en el IDE utilizado para desarrollar la aplicación.
3. Verificar que las clases importadas de los catálogos estén disponibles.
4. Ejecutar la clase `Main`.

Main.java

```
public static void main(String[] argumentos) {  
    PoketerminalApp controladorJuego = new PoketerminalApp();  
    controladorJuego.iniciarAplicacion();  
}
```

Funcionamiento interno

Inicio

`iniciarAplicacion()` muestra la pantalla inicial y luego el menú principal. El menú acepta **W**, **S** y **X** para navegar y confirmar.

Creación de partida

Se solicita un nombre de partida, se ejecutan diálogos introductorios, se pide el nombre del entrenador y se elige un Pokémon inicial. La implementación asigna los identificadores 1, 7 y 4 a las tres opciones disponibles.

Exploración

El jugador se mueve con **W/A/S/D**, entra o interactúa con **X**, abre el menú general con **1-6** y sale de la partida con **Q**. Solo puede desplazarse a casillas que `Mapa.esCasillaPasable( ... )` considera transitables.

Encuentros salvajes

Al caminar sobre hierba alta, el código realiza una comprobación aleatoria; cuando se activa, se genera un Pokémon salvaje y se inicia una batalla mediante `SistemaBatalla`.

Interacción

La tecla **X** detecta puertas de edificios por símbolos y color: Centro Pokémon, Tienda Pokémon o Gimnasio Pokémon. Los edificios son subclases de `Edificio`.

Clases y responsabilidades

Clase	Responsabilidad	Relaciones destacadas

Main	Punto de entrada.	Crea PoketerminalApp.
PoketerminalApp	Control del juego y menús.	Usa Partida, SistemaBatalla, UI y utilidades.
Pokemon	Estado y comportamiento de un Pokémon.	Contiene movimientos, experiencia, vida y estadísticas.
Jugador	Estado del entrenador.	Equipo de Pokémon, medallas, mochila, posición y Pokémonedas.
Partida	Estado persistente de una sesión.	Jugador, ciudades, estadísticas y generación del mundo.
Mapa / MapaCiudad	Representación y validación del mapa.	MapaCiudad extiende Mapa; administra edificios.
Edificio	Abstracción común para construcciones.	CentroPokemon, TiendaPokemon y GimnasioPokemon extienden Edificio.
SistemaBatalla	Ejecuta turnos y acciones de combate.	Interactúa con Jugador, Pokemon y Movimiento.
GestorDeArchivos	Persistencia de partidas y Hall.	Serialización de Partida; archivos .dat y .txt.

[OK] Herencia observada

El UML refleja explícitamente la herencia de **MapaCiudad → Mapa** y de los tres edificios hacia **Edificio**.

Persistencia de información

Las clases `Partida`, `Jugador`, `Pokemon` y otros modelos implementan `Serializable`, lo que permite guardar una partida completa mediante serialización de objetos.

GestorDeArchivos.java

```
String rutaArchivo = partida.getNombrePartida() + ".dat";
try (ObjectOutputStream flujoSalida =
    new ObjectOutputStream(new FileOutputStream(rutaArchivo))) {
    flujoSalida.writeObject(partida);
    return true;
}
```

El Hall de la Fama se conserva en `hall_de_la_fama.txt` y se abre en modo de anexo para registrar nuevas entradas. `filecite turn0file0 L61-L103`

Pruebas recomendadas

Prueba	Resultado esperado
Nueva partida	Solicita nombre, entrenador y Pokémon inicial y genera un mundo.
Movimiento	Las teclas W/A/S/D cambian de posición únicamente en casillas pasables.
Interacción con edificio	X abre el edificio según la puerta detectada.

Equipo	Permite seleccionar, reordenar y liberar Pokémon sin dejar al jugador sin Pokémon.
Guardar/cargar	Se crea o lee el archivo <code>nombre.dat</code> .
Hall de la Fama	Se muestra el contenido si existe el archivo de registros.
Batalla	Se muestran movimientos, daño, cambios y objetos disponibles según el estado del combate.

Manejo de errores

La lectura de partidas captura `IOException` y `ClassNotFoundException`. Las entradas numéricas de varios menús están protegidas mediante `NumberFormatException`. Cuando se produce un error de archivo, el programa informa el problema por consola y devuelve `null` o `false` según la operación. `filecite\turn0file0\L72-L78`

[ERROR] Archivos externos

Una partida se guarda con el nombre solicitado por el usuario más `.dat`. Si se cambia, mueve o elimina el archivo, la carga no podrá recuperar esa partida.

Mantenimiento

Para ampliar el sistema, se recomienda conservar la separación por paquetes. Nuevas entidades del juego deben ubicarse en `modelos`; nuevas construcciones en `mapas.edificios`; reglas de combate en `logica`; y elementos puramente visuales en `ui`.

Los mapas y pixel art contienen estructuras de datos grandes, por lo que cualquier cambio debe probarse directamente en terminal para verificar alineación y caracteres Unicode.

Diagramas UML

El paquete de entrega incluye el diagrama UML en dos formatos:

Archivo editable

[Poketerminal_UML.drawio](#)

Versión PDF

[Poketerminal_UML.pdf](#)

El UML refleja clases, atributos, operaciones, herencia y relaciones principales entre el controlador, modelos, mapas, edificios y sistema de batalla. En las páginas del PDF se observan, por ejemplo, `Main`, `PoketerminalApp`, `SistemaBatalla`, `Pokemon`, `Jugador`, `Partida`, `Mapa`, `MapaCiudad` y los tres edificios. turn1file0L5-L23fileciteL97-L123

Versiones

Versión	Descripción
1.0	Versión de entrega documentada en este manual.

PokeTerminal • Manual Técnico • Documentación de entrega